
building-agents

Build your own agent by building the harness around a model.

01 · WHO I AM

Chase Dovey

I research agentic systems.
Most of my work is building
harnesses around
foundational models.

02 · THE LAB

Average Joes Lab

A private citizen research
lab. Independent research
on agentic systems,
published openly.

03 · THIS TALK

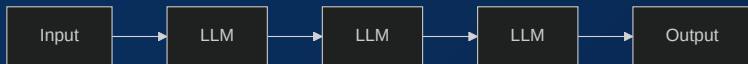
Agent = Model + Harness

The 3 disciplines, and a 10-
module curriculum that
builds quark — a complete
agent harness — from a
single LLM call.

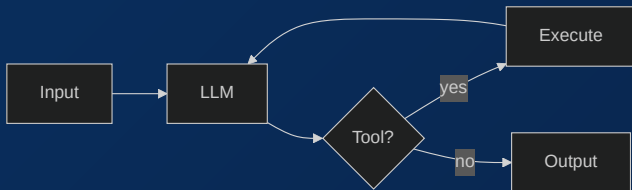
What are agentic systems?

Systems that act on their own. The agency comes from an LLM coordinating calls to reach a goal without supervision.

WORKFLOW · CODE DECIDES THE PATH



AGENT · MODEL DECIDES THE PATH



I focus on agents.

Systems with autonomy over their own control flow.

EXAMPLES IN THE WILD

- Claude Code
- Cursor
- Devin
- Aider
- openclaw

The three disciplines

Three layers stacked. You go through them in order — from nothing to agent-built software.

01 · OUTPUT: CALLABLE MODEL

Model development

A handful of labs train the foundational model.

GPT · Claude · Gemini · Llama

02 · OUTPUT: AN AGENT

Harness engineering

Wrap the model in code, state, tools, loop.

Agent = Model + Harness

Claude Code · Cursor · Codex

← **this talk's focus**

03 · OUTPUT: PRODUCTS BUILT BY AGENTS

Agentic engineering

Use the agent to build other software, products, agents.

openclaw · vibe coding's pro
cousin

Discipline 1 · Model development

A handful of labs (Anthropic, OpenAI, Google, Meta) train foundational models. The output is a service you call by API.

ARCHITECTURE · HOW IT ANSWERS

PRE · NOT LEARNED

Input layer · your prompt arrives



Tokenizer · text → number IDs (BPE)



NEURAL NETWORK · LEARNED DURING TRAINING

Embedding · IDs → "meaning vectors"



Positional encoding · adds word-order info (RoPE)



Transformer block × 30-100

the "thinking" — every word looks at every other, refining meaning over and over



LM head · score every possible next word



POST · NOT LEARNED

Output layer · sample one word, return text → loop

TRAINING · HOW IT LEARNED (SHAPES THE NEURAL NETWORK)

1

Pretraining

Next-token prediction over trillions of web tokens — produces the base model.

2

Mid-training

Continued pretraining on curated data — sharpens code, math, reasoning.

3

Supervised Fine-Tuning (SFT)

Instruction/response pairs — teach the model to follow instructions.

4

Preference tuning · RLHF / DPO / GRPO

Human-rated comparisons — align outputs to be helpful, honest, harmless.

5

Constitutional AI / RLAI

AI feedback against written principles — scales alignment past human-only labeling.

6

Reasoning RL · GRPO + verifiable rewards

Math/code with rule-based rewards — trains explicit chain-of-thought (R1, o-series).

We don't teach this. The harness layer assumes it's already happened **upstream**.

Discipline 2 - Harness engineering

Agent = Model + Harness

The harness is every piece of code, configuration, and execution logic that isn't the model itself.

THE HARNESS WE BUILD: QUARK'S 9 COMPONENTS

Model interface

A streamed call, read event by event — so the harness can act between events.

Control flow

One `while True` loop, bound to an environment (the terminal).

Body · tools

One bash tool. Its reach is the whole system; know-how lives in the prompt.

Self model

The system prompt: self, world, other selves, and how to use the body.

Working memory

When the context window overflows: drop the oldest turns, summarize the rest.

Long-term memory

A markdown file the model writes and greps with its own body.

Interrupts

ESC stops speech or kills a command — and the conversation stays valid.

Self-knowledge

The harness embeds its own source code in the prompt.

Caching

A byte-stable system prompt, cached — the big prefix is paid for once.

10 modules build them one at a time — ending at quark, byte for byte. This is the talk's focus.

Discipline 3 · Agentic engineering

Once you have an agent — a model wrapped in a harness — what do you do with it?

A · OUTWARD

Develop other products

Point the agent at the next codebase. Ship features, build infrastructure, author tooling.

Example: Peter Steinberger built openclaw by directing existing coding agents, then embedded a harness inside it.

B · RECURSIVE

Develop the agent itself

Point the agent at its own curriculum. Write a new module, refactor a component, raise the evals.

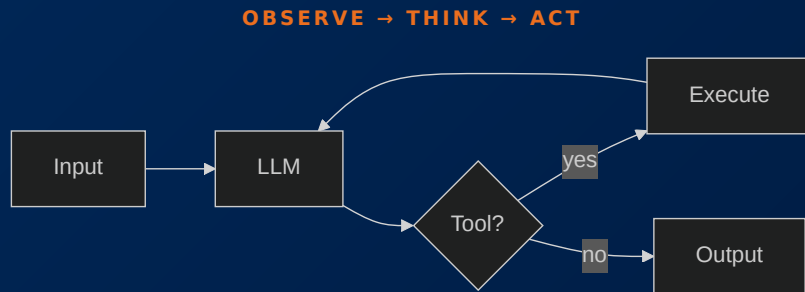
This repo and deck are built that way: Claude Code (a harness) running on Claude, driven by me.

Vibe coding's disciplined cousin.

Same fundamental move — have AI write the code — but with thought about what to ask, what tools to provide, how to verify, how to ship. **You're the human orchestrating agents to do the engineering.**

Module 1 - What is an agent?

concept · modules/01-what-is-an-agent/ → 01_toy.py



Three primitives: an **LLM call** (the model), a **tool** and a **loop** (the harness). The toy uses one tool — bash — which quark keeps as its whole **body**.

Module 2 - An LLM call

harness component: model interface · → 02_stream.py

```
with client.messages.stream(model=MODEL, max_tokens=4096, messages=working_memory) as stream:
    for ev in stream:
        if ev.type == "content_block_delta" and hasattr(ev.delta, "text"):
            sys.stdout.write(ev.delta.text); sys.stdout.flush()
    saying = stream.current_message_snapshot
```

Stream every call and read it **event by event**: the event loop is where the harness gets control — later, to check for ESC. `current_message_snapshot` is exactly what arrived, even if we stop early.

Module 3 - Add a loop

harness component: control flow · → 03_loop.py

```
while True:
    with client.messages.stream(...) as stream: ...          # Module 2
    print()
    working_memory.append({"role": "assistant", "content": saying.content})
    if not chat: break                                     # one-shot: done
    u = next(filter(str.strip, iter(lambda: input("\n> "), None))) # re-prompt on blank input
    if u == "/q": break
    working_memory.append({"role": "user", "content": u})
```

The API is stateless; `working_memory` is the agent's mind. The terminal is just quark's environment — **the same loop binds to a web socket, a chat app, or a robot.**

Module 4 - Add a body

harness component: `tools` · → `04_body.py`

01 · ONE TOOL

bash is the body

Its reach is the whole system.
Know-how lives in the prompt,
not in more tools.

02 · ONE LOOP

No inner loop

If the reply has tool calls, act and
go round again. No calls → the
turn is over → ask the human.

03 · INVARIANTS

Never wedge, never misfire

Drain the pipe as it fills (no 64KB
deadlock). Bounded final drain.
Never run a command cut off by
`max_tokens` .

Every `tool_use` gets its `tool_result` — even the ones that never ran: `[your doing ... never reached the world]` .

Module 5 - Add a self model

harness component: system prompt · → 05_self_model.py

SELF MODEL

What it is

Identity, mind (the context window), body (bash), loop (observe → think → act).

WORLD MODEL

Where and when

Environment, working directory, time — things only the harness can tell it.

OTHER SELVES

Who it's with

Humans reach it with text; it reaches them by speaking through its body.

Body Operations teaches what a toolkit would hard-code: one focused act per response, an escalation gradient (pipes → `python -c` → scripts → installs), and a grounding gradient (mind → world → ask).

Module 6 - Add working memory

harness component: context management · → 06_working_memory.py

01 · OVERFLOW

Wait for the 400

Reactive: no tokenizer, no budget. `prompt is too long` →
`drop += 1`.

02 · SLICE

Cut at turn boundaries

Drop the oldest `drop` turns.
Slicing at a user-text message
never orphans a tool result.

03 · SUMMARIZE

Replace the mind

The model writes a gist;
`working_memory = [summary]`.
Retry until it's non-empty.

Module 7 - Add long-term memory

harness component: persistence · → 07_long_term_memory.py

```
## 2026-10-01 14:02:11
```

- Chase prefers short answers
- project tests run with: `uv run pytest -q`

Zero lines of code. A format contract, a write recipe, and read "moves" in the prompt — the model keeps `.quark/memory/memory.md` with its own body. This is system prompt learning: the model is frozen; the harness gets smarter.

Module 8 - Add interrupts

harness component: human control · → 08_interrupts.py

NOTICE

An observer thread

cbreak mode, raw byte reads. A lone `\x1b` sets `interrupt`; arrow-key sequences are ignored.

STOP

At every yield point

Break the stream (closes the connection). `killpg` the command's whole process group.

CLOSE OUT

Keep history valid

Pair every tool call, keep what was said, then `[other self interrupted what you were ...]`.

Module 9 - Add self-knowledge

harness component: self-knowledge · → 09_self_knowledge.py

```
def mechanics(): return "\n".join('def system(): return "<system prompt redacted ...>"  
    if l.startswith("def system():") else l  
    for l in open(__file__).read().split("\n"))
```

The harness embeds its own source in the prompt. The model can explain its own behavior — and it can never be out of date. Same vocabulary in prompt and code: *mind, body, doing*.

Module 10 · Add caching

harness component: performance · → quark.py

01 · MARK IT

cache_control

System prompt becomes one block marked `ephemeral`: tools + system cached, read back at ~0.1x.

02 · KEEP IT STABLE

Date, not time

A timestamp would change every second and silently miss. The model gets the date — and `date` via its body.

The curriculum's destination: `examples/quark.py` — 81 lines, byte for byte.

Module 11 · Add a sandbox

harness component: execution environment · beyond quark · → 11_sandbox.py + Dockerfile.sandbox

01 · CONTAIN

One body, one box

`docker exec` instead of a host shell. No network, read-only system, caps dropped, one shared directory.

02 · INTERRUPT

Kill where it lives

Killing the host-side client leaves the command running. Record the group ID,
`kill -9 -- -pgid` inside.

03 · ALIGN

Tell the truth

New body, new self model:
`/workspace`, no network, nothing to install — "say so" instead of failing.

A toolkit leaks around a sandbox. **One body means sandboxing one tool contains everything the agent can do.**